Data Structures and Algorithms - Lab

Topic

Understanding the Concept of Queue (First In, First Out) and Circular Queue through Basic Applications

Objectives

- Understand the purpose of Queue and Circular Queues and their First In, First Out (FIFO) approach.
- Explore the use of enqueue and dequeue operations in Queue and Circular Queue.
- Practice converting and evaluating expressions using Queues and Stacks for expression manipulation tasks.
- Familiarize with menu-driven programs to apply Queue concepts in simulations.

Outcomes

1. Develop a comprehensive understanding of creating and using Queue and Circular Queue structures to manage ordered data efficiently.
2. Gain experience implementing enqueue and dequeue operations and understanding their role in managing data in a FIFO manner.
3. Enhance problem-solving abilities by designing functions that meet specific operational requirements, such as customer management and expression evaluation.
4. Gain expertise in expression evaluation by converting infix expressions to prefix expressions and calculating their results using Queue and Stack data structures.

Content

The Queue class has the following attributes:

1. `T *arr` - This is a pointer to the array that holds the elements in the queue.
2. `int maxSize` - This is the maximum size of the queue.
3. `int currentSize` - This is the current size of the queue.

There are several pure virtual functions inside the Queue class:

1. `void enqueue(T data)` - This function is used to add data to the back of the queue.
2. `T dequeue()` - This function is used to remove and return the data at the front of the queue.
3. `T peek()` - This function is used to get the data at the front of the queue without removing it.
4. `bool isFull()` - This function is used to check if the queue is full.
5. `bool isEmpty()` - This function is used to check if the queue is empty.

The Queue class also has a constructor and a destructor:

1. `Queue(int maxQueueSize)` - This constructor takes the maximum size of the queue as a parameter and initializes the queue.
2. `~Queue()` – The destructor makes sure that there is no memory leakage.

To use the `Queue<T>` class, you need to create your own class, `MyQueue<T>`, which inherits from the `Queue<T>` class. `MyQueue<T>` should be a template class that takes a type `T` as a parameter.

Task Instructions

Students are required to complete the following tasks during lab time. Create a public repository on your GitHub accounts. The name of the repository should be **Circular-Queue** . All files should be uploaded to this repository, including the header and cpp files. Please note that the name of class-related files should be the same as the class name. If there is only 1 task, we can name the main file Task_1.cpp or Task.cpp. We recommend you work in .h files for classes only since we must work on templates.

Task 1

In a bank, customers stand in line to be served. Each customer has a **name** and a **transaction type** (e.g., Deposit, Withdrawal, Loan Inquiry). The first customer to arrive is the first to be served (FIFO principle).

Task:

1. Implement a **queue using a circular array** where each element is a structure containing:
    - string name → Customer name
    - string transaction → Transaction type
2. Implement the following functions:
    - enqueue(Customer c) → Add a new customer to the queue when they arrive.
    - dequeue() → Remove and return the first customer when they are served.
    - isEmpty() → Check if the queue is empty.
    - isFull() → Check if the queue is full.
    - display() → Show all customers currently waiting, with their names and transaction types.
3. Write a **menu-driven program** with options:
    - Add a customer (enqueue) → user enters name and transaction type.
    - Serve a customer (dequeue) → display which customer was served and their transaction type.
    - Display all waiting customers.
    - Show total number of customers currently waiting.
    - Exit.
4. Demonstrate **overflow** (when queue is full) and **underflow** (when trying to serve from an empty queue) by printing appropriate error messages.

Task 2

In a cinema hall, there are a limited number of seats available for customers. Customers request tickets, and their requests are handled in First In, First Out (FIFO) order. When a customer cancels a booking, the available space should be reused for the next customer.

A Circular Queue can be used to manage ticket bookings efficiently by reusing empty positions in the array.

Task for Students

1. Implement a Circular Queue using an array to manage ticket bookings.
2. Each booking should contain:

- customerID – Unique customer ID.
- customerName – Name of the customer.
- numberOfTickets – Number of tickets booked.

3. Implement the following functions:
    - enqueue(Booking b) → Add a new booking to the queue.
    - dequeue() → Remove and return the booking at the front.
    - peek() → Display the booking at the front without removing it.
    - isFull() → Check whether the queue is full.
    - isEmpty() → Check whether the queue is empty.
    - display() → Display all current bookings.

Menu-driven program

1. Book tickets.
2. Cancel the first booking.
3. Display the next booking.
4. Display all bookings.
5. Show the total number of bookings.
6. Exit.

Task 3

In an office, multiple users send documents to a shared printer. The printer processes documents in the order they are received. Once a document is printed, it is removed from the queue, and a new print job can be added.

A Circular Queue can be used to manage print jobs efficiently and reuse the available positions in the queue.

Task for Students

1. Implement a Circular Queue using an array to manage printer jobs.
2. Each print job should contain:
    - jobID – Unique print job ID.
    - userName – Name of the user who submitted the job.
    - pages – Total number of pages to print.
3. Implement the following functions:

- enqueue(PrintJob p) → Add a new print job to the queue.
- dequeue() → Remove and return the completed print job.
- peek() → Display the next print job to be processed.
- isFull() → Check whether the queue is full.
- isEmpty() → Check whether the queue is empty.
- display() → Display all pending print jobs.

Menu-driven program

1. Submit a new print job.
2. Process the next print job.
3. Display the next print job.
4. Display all pending print jobs.
5. Show the total number of pending print jobs.
6. Exit.